

RGS_GATE_SubSystemFirmware

Consolidating three MPLAB X projects into a single repository

Working notes · 7 August 2026 · Unicycle Systems Ltd

1. Purpose and scope

Three MPLAB X projects (`Bootloader_V2.X`, `RGS_BringUp.X`, `PuttingGate.X`) currently live in separate Git repositories under an unversioned parent folder. A growing set of shared source and header files needs to stay synchronised across all three, which separate repositories make awkward.

The plan: a single Git repository at `C:\RGS_GATE_SubSystemFirmware` containing the three project folders as siblings, plus a `CommonFiles` directory holding the shared code. Each project references the shared files by relative path rather than holding its own copy.

Target structure

<code>C:\RGS_GATE_SubSystemFirmware\</code>	<code><- single git root</code>	<code>CommonFiles\</code>	<code>header\</code>	<code>source\</code>	<code>Bootloader_V2.X\</code>	<code>RGS_BringUp.X\</code>	<code>PuttingGate.X\</code>
---	------------------------------------	---------------------------	----------------------	----------------------	-------------------------------	-----------------------------	-----------------------------

Moving to the root of `C:\` is worth doing beyond tidiness: MPLAB X `nbproject` paths get deep, and the previous location consumed characters against the 260-character Windows path limit for no benefit.

2. Trade-offs accepted

Genuine sharing means a change made for one project silently affects the other two — the coupling becomes real rather than aspirational. Two mitigations:

- Keep `CommonFiles` device-agnostic. Let each project supply its own configuration header on the include path (e.g. `#include "subsystem_config.h"`, defined per project).
- Build all three before pushing any change that touches `CommonFiles`. A small script that batches the three builds pays for itself quickly.

Tagging also changes: `v1.2` is no longer meaningful for a single project. Use prefixed tags from the outset — `bootloader-v1.2.0`, `puttinggate-v1.2.0`.

3. Why `git subtree` rather than copying

`git subtree add` performs real copying — no symlinks, no submodules, no ongoing link to the source repository. After it runs, the files are ordinary tracked files in the new repo, and deleting the old repositories would break nothing.

What it adds over `cp -r` is **history**. It fetches the commits from the source repository, rewrites them so every path is prefixed with the target directory, and merges that rewritten history in. `git log` and `git blame` on the migrated files continue to work across the whole history. That is the entire reason for using it.

Full history vs. `--squash`. Adding `--squash` collapses each import into a single commit, giving a tidier log at the cost of all per-file history before the merge. Full history is the right choice here: `git blame` on firmware gets used far more often than a tidy log, and the old repositories stop being a place worth looking once migration is complete.

4. Migration procedure

Run in **Git Bash** (not `cmd`), with MPLAB X closed.

Step 0 — Pre-flight checks

Determine the branch name in each source repository, and confirm nothing is uncommitted. `git subtree` reads from committed history, not the working tree, so uncommitted work will not migrate.

```
cd /c/Projects_Firmware/Bootloader_V2.X
git branch --show-current
git status --short

cd /c/Projects_Firmware/RGS_BringUp.X
git branch --show-current
git status --short

cd /c/Projects_Firmware/PuttingGate.X
git branch --show-current
git status --short
```

Git Bash also prints the current branch in brackets at the end of the prompt, which answers the branch question without running anything.

Interpreting `git status --short`. Empty output is the ideal result. Otherwise, the two-character prefix matters — left column is staged, right column is unstaged:

Prefix	Meaning	Action
??	Untracked (usually <code>build/</code> , <code>dist/</code> , <code>nbproject/private/</code>)	Benign — proceed
M	Modified, not staged	Commit before importing
M / MM	Staged, possibly with further edits	Commit before importing
A	New file staged, not committed	Commit before importing
D	Deleted in working tree, still tracked	Confirm intent, then <code>git rm</code> or <code>git restore</code>

Rule of thumb: if every line starts with `??` and points at build artefacts, proceed. Any letter in either column means commit first. To filter out untracked noise: `git status --short --untracked-files=no` — empty output there means it is definitely safe.

Step 1 — Create the new repository

```
cd /c
git init -b main RGS_GATE_SubSystemFirmware
cd RGS_GATE_SubSystemFirmware
```

`-b main` sets the branch name explicitly rather than relying on the Git default, which varies by version and configuration.

Step 2 — Empty root commit

```
git commit --allow-empty -m "Initial commit"
```

Scaffolding only: `git subtree add` refuses to run against a repository with no history. The commit sits at the root of the history permanently and harmlessly.

Step 3 — Import the three projects

The trailing branch name must match that repository's actual branch. A mix of `master` and `main` across the three is fine — each command is independent, and the distinction disappears once everything is merged onto the monorepo's `main`. Adjust the names below to match what Step 0 reported.

```
git subtree add --prefix=Bootloader_V2.X /c/Projects_Firmware/Bootloader_V2.X master
git subtree add --prefix=RGS_BringUp.X /c/Projects_Firmware/RGS_BringUp.X main
git subtree add --prefix=PuttingGate.X /c/Projects_Firmware/PuttingGate.X master
```

If it fails with fatal: couldn't find remote ref main — the branch name is wrong for that repository; substitute `master`. Nothing is written by a failed attempt, as the fetch stops before touching the new repository. No cleanup needed.

Run one at a time and verify each before the next. A success ends with something like `Added dir 'Bootloader_V2.X'`.

```
ls Bootloader_V2.X
git log --oneline -- Bootloader_V2.X | tail -3
```

The second command showing the *oldest* commits from the original repository is the proof that history came across, not just files.

Step 4 — Create the common directories

Git does not track empty directories, so seed them:

```
mkdir -p CommonFiles/header CommonFiles/source
touch CommonFiles/header/.gitkeep CommonFiles/source/.gitkeep
```

Step 5 — Root `.gitignore`

```

cat > .gitignore << 'EOF'
# MPLAB X build output
build/
dist/
debug/
*.production.hex
*.debug.hex
*.production.map
*.debug.map

# Per-machine MPLAB state (absolute paths, debugger serials)
nbproject/private/
**/nbproject/private/

# Generated makefiles
**/nbproject/Makefile-*.mk
**/nbproject/Makefile-genesis.properties
**/nbproject/Makefile-impl.mk
**/nbproject/Makefile-local-*.mk
**/nbproject/Makefile-variables.mk

# Claude Code
.claude/settings.local.json
CLAUDE.local.md

# OS
Thumbs.db
Desktop.ini
EOF

```

`nbproject/private/` is the important exclusion — it holds absolute paths, tool serial numbers and per-machine debugger configuration, and produces noisy conflicts if committed.

Step 6 — Root `.gitattributes`

Without this file, line-ending behaviour is governed by each machine's `core.autocrlf` setting: invisible, easy to get wrong, and different between a Git Bash install and a TortoiseGit one. `.gitattributes` makes the repository's rules explicit and self-contained.

The practical failure mode is `configurations.xml`. MPLAB rewrites it constantly; if line endings flip, the diff shows every line changed, making real project changes impossible to review and any merge a mess.

```

cat > .gitattributes << 'EOF'
* text=auto

*.c text
*.h text
*.s text
*.inc text
*.xml text
*.mk text

*.bat text eol=crlf
*.cmd text eol=crlf
*.sh text eol=lf

*.hex -text
*.elf binary
*.cof binary
*.pdf binary
EOF

```

`* text=auto` normalises to LF in the repository and checks out CRLF on Windows. The explicit `.bat` and `.sh` rules matter because those files need correct endings to execute at all. Intel HEX is marked `-text` so it passes through byte-for-byte, which is safest if it ever meets a checksum or a fussy programmer.

Step 7 — Commit and verify

```

git add .gitignore .gitattributes CommonFiles
git commit -m "Add monorepo structure, gitignore and gitattributes"

ls
git log --oneline | head -20
git log --oneline -- Bootloader_V2.X | tail -5

```

Step 8 — Untrack anything already committed that should not be

A new `.gitignore` does not untrack files the old repositories already had under version control:

```

git rm -r --cached --ignore-unmatch */build */dist */nbproject/private
git status --short
git commit -m "Stop tracking build output and per-machine MPLAB state"

```

Check `git status` before committing — confirm it is removing only what is expected.

Step 9 — Push to GitHub

Create an empty repository on GitHub (no README, no `.gitignore` — both already exist locally), then:

```

git remote add origin https://github.com/<org>/RGS_GATE_SubSystemFirmware.git
git push -u origin main

```

Open each project from the new location and build all three *before* changing anything, establishing a known-good baseline. Only then move the shared files into `CommonFiles` and repoint the projects.

Rollback. Keep `C:\Projects_Firmware` intact until all three build cleanly from the new location. Rename it `C:\Projects_Firmware_OLD` rather than deleting. Archive the three GitHub repositories read-only once confident, so nothing gets committed to the wrong one at 11pm.

5. MPLAB X: how the shared files get referenced

MPLAB X stores its file list in `<project>.X\nbproject/configurations.xml` as `<itemPath>` entries, which may be absolute or relative.

- When using **Add Existing Item** on a file outside the project folder, choose **Relative**. Auto usually picks relative for a reachable path on the same volume, but verify rather than trusting it.
- Relative yields `../CommonFiles/source/foo.c` in the XML, which resolves identically on any machine. Absolute embeds `C:/Users/...` and makes the repository effectively unclonable.
- Include paths: *Project Properties* → *xc16-gcc* → *Preprocessing and messages* → *Include directories* should read `../CommonFiles/header`. **Check every build configuration**, not only the active one — debug and release configs are easy to miss.
- Add the files as links, not copies. Create a logical folder named `Common` in each project so it is visually obvious which sources are shared and which are project-private.

6. Regenerating the makefiles

MPLAB X does not build anything itself: it generates makefiles from `configurations.xml`, then calls `make`. Those `Makefile-*.mk` files are derived artefacts, like object files — not source.

Approach	Consequence
Ignore them (block left in <code>.gitignore</code>)	A fresh clone has no makefiles, so a bare <code>make</code> fails until they are generated. One-off generation step per clone — either open the project in MPLAB X once, or run <code>prjMakefilesGenerator</code> . Not a permanent restriction to manual builds.
Commit them (block removed)	Clone-and- <code>make</code> works immediately. Cost is churn: MPLAB rewrites them on file additions, compiler setting changes and version upgrades, and they contain machine- and version-specific paths — so diff noise and conflicts on files never edited by hand.

Decision: ignore them. Three projects, effectively one developer. A generation call run twice ever is nothing; conflicts on generated files would be a recurring annoyance. If CI arrives later, the clean answer is a build script that calls `prjMakefilesGenerator` first, not committed generated output.

Regeneration script

`prjMakefilesGenerator.bat` handles one project at a time, so loop over the three. Worth saving as `regen-makefiles.sh` at the repository root and committing — it will be needed again after any fresh clone, and drops straight into CI later.

```
#!/usr/bin/env bash
# Regenerate MPLAB X makefiles for all three projects.
# Run from the repository root in Git Bash.

MPLAB="/c/Program Files/Microchip/MPLABX/v6.20/mplab_platform/bin/prjMakefilesGenerator.bat"

for p in Bootloader_V2.X RGS_BringUp.X PuttingGate.X; do
    echo "=== $p ==="
    "$MPLAB" "$(pwd -W)/$p"
done
```

`pwd -W` yields the Windows-style path (`C:/RGS_GATE_...`), which the batch file requires — a MinGW-style `/c/...` path may not resolve. Before looping, confirm the installed MPLAB version and that generation works on a single project:

```
ls "/c/Program Files/Microchip/MPLABX/"
ls Bootloader_V2.X\nbproject/Makefile-*.mk
```

7. Claude Code project portability

Separate topic from the migration, but relevant to how the new repository is set up.

Commit these

The `.claude/` directory in a repository is the "project" scope and is designed to be shared:

- `CLAUDE.md` at the repository root (or `.claude/CLAUDE.md`) — build commands, toolchain quirks, conventions
- `.claude/settings.json` — permission rules, hooks, model choice
- `.claude/commands/`, `.claude/agents/`, `.claude/skills/`
- `.mcp.json` at the repository root — project-scoped MCP servers live here, not inside `.claude/`, which catches people out

Keep these out

- `.claude/settings.local.json` — personal per-machine layer, and where "don't ask again" permission approvals are stored. When Claude Code writes it, the pattern is added to the *global* git excludes file, not the repository's `.gitignore` — so a hand-created file, or a clone on a machine lacking that global exclude, can slip into a commit. List it in the repository `.gitignore` explicitly.
- `CLAUDE.local.md` — personal project notes.

Portability traps

1. **Absolute paths** in hook commands, `additionalDirectories` or statusline scripts. Push machine-specific values into `settings.local.json` or drive them from environment variables in the `env` block.
2. **Shell assumptions.** Hooks run through `/bin/sh` on Linux and `cmd` on Windows.
3. **Line endings and exec bits** on committed hook scripts — covered by the `.gitattributes` above.
4. **Toolchain paths.** MPLAB X and XC16 invocation differs enough between machines that hard-coding into `settings.json` will irritate; reference by environment variable.

Sessions do not travel. Session transcripts and conversation history live under `~/.claude/` on each machine, keyed by directory path — not in the repository's `.claude/`. Committing the project folder gives portable *configuration*, not portable *sessions*.

Monorepo layout for Claude Code

A monorepo suits it well: a root `CLAUDE.md` describing the shared architecture and the "rebuild all three when CommonFiles changes" rule, plus a `CLAUDE.md` inside each `.X` folder for project-specific detail. Child files are pulled in on demand when files in those directories are read, so the right context arrives without bloating every session.

Also worth versioning `~/.claude/` (personal `CLAUDE.md`, `settings.json`, agents, commands) as its own separate private repository — that makes the personal setup portable independently of any project.

After cloning on a second machine, `/status` lists every settings source actually loaded. A file with broken JSON simply will not appear, which is the fastest way to spot a configuration that is silently not taking effect.

8. Outstanding

- Decide what genuinely belongs in `CommonFiles` versus what merely looks shared — the split has not been done yet.
- Reprint each project's item paths and include directories, per section 5.
- Write the batch-build script covering all three projects.
- Set up prefixed tagging convention before the first release from the new repository.